Explainable Breast Cancer Detection using Ensemble Learning

Individual Contribution Report

EPICS Phase – II

Submitted by

Ujjwal Mishra
Register Number: 23BEC10079

in partial fulfillment of the requirements for the degree of

Bachelor of Technology
Electronics and Communication Engineering

VIT Bhopal University
Kothrikalan, Sehore
Madhya Pradesh

March 2026

Abstract

Breast cancer continues to be a major health concern worldwide, and catching it early can save lives. Our team built a machine learning system that not only detects cancer accurately but also explains its reasoning – important for any medical application where doctors need to trust and understand the output.

We tested out four different classifiers – Logistic Regression, Decision Trees, SVM, and Random Forest – and combined them with soft voting. My job was specifically implementing these individual algorithms and figuring out which configurations worked best.

We focused heavily on making the system interpretable, with confidence scores and feature importance analysis. In the end, Logistic Regression and SVM both hit 98.25% accuracy with ROC-AUC scores above 0.99 – pretty impressive results for classical ML methods.

Keywords: Breast Cancer Detection, Machine Learning, Classification Algorithms, Support Vector Machine, Random Forest, Logistic Regression

Contents

1 MY CONTRIBUTION **1**
- 1.1 Overview . . . 1
- 1.2 Algorithm Implementation . . . 1
  - 1.2.1 Logistic Regression . . . 1
  - 1.2.2 Decision Tree Classifier . . . 1
  - 1.2.3 Support Vector Machine (SVM) . . . 2
  - 1.2.4 Random Forest Classifier . . . 2
- 1.3 Model Training and Optimization . . . 3
  - 1.3.1 Training Process . . . 3
  - 1.3.2 Hyperparameter Selection . . . 3
- 1.4 Model Comparison . . . 3
- 1.5 Algorithm Analysis . . . 4
  - 1.5.1 Why Logistic Regression Excelled . . . 4
  - 1.5.2 Why SVM Performed Well . . . 4
  - 1.5.3 Why Decision Tree Underperformed . . . 4

2 PROJECT OVERVIEW **5**
- 2.1 Problem Statement . . . 5
- 2.2 System Architecture . . . 5
- 2.3 Results Achieved . . . 5

3 TEAM CONTRIBUTIONS **6**
- 3.1 Team Members and Their Roles . . . 6
- 3.2 Collaboration . . . 6

4 LEARNING OUTCOMES **7**

5 CONCLUSION **8**

REFERENCES **9**

1. MY CONTRIBUTION

1.1 Overview

My main job on this project was **implementing the machine learning algorithms** we used for classification. I got to work hands-on with four different classifiers, tuning them and figuring out what worked best for our breast cancer detection problem.

1.2 Algorithm Implementation

I implemented four classification algorithms. Here's how each one went:

1.2.1 Logistic Regression

This was our baseline – simple but effective. It models the probability of malignancy using a sigmoid function:

$$P(y = 1|x) = \sigma(w^T x + b) = \frac{1}{1 + e^{-(w^T x+b)}} \tag{1}$$

What I actually set up:

- Added L2 regularization (C=1.0) to prevent overfitting
- Bumped max_iter to 5000 because it wasn't converging with the default
- Used LBFGS optimizer (works well for smaller datasets)
- Hit 98.25% accuracy – honestly surprised how well this simple model did

Listing 1: Logistic Regression Implementation

```
from sklearn.linear_model import LogisticRegression

lr_model = LogisticRegression(
    max_iter=5000,
    C=1.0,
    random_state=42
)
lr_model.fit(X_train, y_train)
```

1.2.2 Decision Tree Classifier

I included Decision Tree mostly for interpretability – you can actually trace through its decisions:

- Used Gini impurity for splits (standard choice)
- Let it grow to full depth to see what it could do
- Only got 91.23% accuracy – definitely the weakest performer

The Decision Tree's rules are easy to understand, but it struggled compared to other methods. Probably overfitting on training data.

1.2.3 Support Vector Machine (SVM)

SVM with RBF kernel was interesting to implement. The kernel maps data to higher dimensions:

$$K_{RBF}(x_i, x_j) = \exp\left(-\gamma \|x_i - x_j\|^2\right) \tag{2}$$

My configuration:

- RBF kernel (handles non-linear boundaries if they exist)
- C=1.0 for regularization
- Had to enable probability=True for the ensemble to work
- gamma='scale' lets sklearn pick a reasonable value automatically
- Tied with Logistic Regression at 98.25%

Listing 2: SVM Implementation

```
from sklearn.svm import SVC

svm_model = SVC(
    kernel="rbf",
    C=1.0,
    probability=True,
    random_state=42
)
svm_model.fit(X_train, y_train)
```

1.2.4 Random Forest Classifier

Random Forest is basically a bunch of decision trees voting together:

$$\hat{y} = \text{mode}\{h_1(x), h_2(x), ..., h_B(x)\} \tag{3}$$

where each $h_b(x)$ is one tree's prediction.
What I configured:

- 100 trees (n_estimators=100) – standard starting point
- Bootstrap sampling for diversity
- No depth limit – let trees grow fully
- Got 95.61% accuracy – decent but not top tier

Listing 3: Random Forest Implementation

```
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
rf_model.fit(X_train, y_train)
```

1.3 Model Training and Optimization

1.3.1 Training Process

I set up a training pipeline that:

1. Takes in the preprocessed data from Suryanshi's work
2. Initializes each model with the hyperparameters I chose
3. Fits each model to training data
4. Saves them for evaluation and ensemble combination

1.3.2 Hyperparameter Selection

I experimented with different settings before settling on these:

Table 1: Final Hyperparameter Configuration

Model	**Parameter**	**Value**
2*Logistic Regression	max_iter	5000
	C	1.0
2*Decision Tree	max_depth	None
	min_samples_split	2
3*SVM	kernel	RBF
	C	1.0
	gamma	scale
2*Random Forest	n_estimators	100
	max_depth	None

1.4 Model Comparison

Here's how all four models performed:

Table 2: Model Performance Comparison

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.9825	0.9861	0.9861	0.9861	0.9954
SVM	0.9825	0.9861	0.9861	0.9861	0.9950
Random Forest	0.9561	0.9589	0.9722	0.9655	0.9939
Decision Tree	0.9123	0.9559	0.9028	0.9286	0.9157

Interesting findings:

- Logistic Regression and SVM tied for best – didn't expect the simplest model to win
- All models except Decision Tree had ROC-AUC above 0.99
- Decision Tree was weakest but gives the most interpretable rules
- Random Forest was middle-of-the-pack – good but not great

1.5 Algorithm Analysis

I spent some time thinking about why the results turned out this way:

1.5.1 Why Logistic Regression Excelled

- Suryanshi's preprocessing (especially scaling) really helped – linear models love normalized data
- Turns out the classes are pretty linearly separable after all
- The regularization prevented overfitting on our 455 training samples

1.5.2 Why SVM Performed Well

- RBF kernel can find non-linear boundaries, but honestly didn't need to here
- 30 features is a nice sweet spot for SVM – not too few, not too many
- Margin maximization gives robust classification

1.5.3 Why Decision Tree Underperformed

- Without pruning, it memorized the training data (overfitting)
- Small changes in data lead to completely different trees
- Can't capture smooth boundaries – everything is axis-aligned splits

2. PROJECT OVERVIEW

2.1 Problem Statement

Our goal was to build a system that distinguishes benign from malignant tumors, provides confidence scores, and explains its decisions. In healthcare, you can't just have a black box – doctors need to understand and trust the reasoning.

2.2 System Architecture

We organized the project into layers:

1. **Data Layer**: Loading and preprocessing
2. **Model Layer**: The classification algorithms (my focus area)
3. **Ensemble Layer**: Soft voting combination
4. **Output Layer**: Metrics and visualizations

2.3 Results Achieved

Final results were solid:

- LR and SVM both hit 98.25% accuracy
- ROC-AUC above 0.99 for the best models
- 98.61% recall – catching almost all actual cancer cases
- Cross-validation confirmed these weren't fluke numbers

3. TEAM CONTRIBUTIONS

3.1 Team Members and Their Roles

- **Bhavya Sharma**: Found the dataset and explained the medical stuff
- **Suryanshi Ranawat**: Data preprocessing and exploratory analysis
- **Ujjwal Mishra (Me)**: Implemented the ML algorithms
- **Medhansh Singh Kushwaha**: Built the ensemble and evaluation system
- **Soumyadeep Chakravarti**: Coordinated everything and wrote it up

3.2 Collaboration

The team worked well together. My models fed directly into Medhansh's ensemble, so we had to coordinate closely on interfaces. Suryanshi's preprocessing made my life easier – the data came in clean and ready to train on.

4. LEARNING OUTCOMES

Here's what I learned from this project:

1. **ML Algorithms**: I really understand these classifiers now – not just how to use them, but the math behind them and when each one makes sense
2. **Model Selection**: Learned that you can't just pick an algorithm randomly – you need to understand your data and problem
3. **Hyperparameter Tuning**: Got practical experience with what different parameters actually do and how to tune them
4. **scikit-learn**: Way more comfortable with the library now, especially the classifier APIs
5. **Algorithm Analysis**: Learned to think about *why* models perform differently, not just what the numbers are
6. **Healthcare ML**: Understood that in medical applications, you care more about recall (not missing cancer) than pure accuracy

5. CONCLUSION

My job on this project was implementing the core ML algorithms – Logistic Regression, Decision Tree, SVM, and Random Forest. I got to experiment with different configurations and figure out what worked best for our cancer detection problem.

The results were honestly a bit surprising. I expected the more complex methods (SVM, Random Forest) to dominate, but simple Logistic Regression tied for the best performance. It taught me that fancy algorithms aren't always better – sometimes clean data and a simple model is all you need.

Each algorithm has its place though. Logistic Regression and SVM gave us the best accuracy, but Decision Tree's rules are easy to explain to non-technical people. Random Forest gave decent performance with built-in feature importance. They all contributed something to the final ensemble.

This EPICS project gave me real hands-on experience with ML implementation. It's one thing to learn about algorithms in class – it's another to actually build them, tune them, and see why they behave the way they do. That practical experience is going to be valuable going forward.

REFERENCES

References

[1] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

[2] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *JMLR*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: `https://scikit-learn.org`

[4] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

[5] S. Chakravarti et al., "EPICS Project: Explainable Breast Cancer Detection," GitHub, 2026. [Online]. Available: `https://github.com/Soumyadeep-Chakravarti/EPICS_Project`